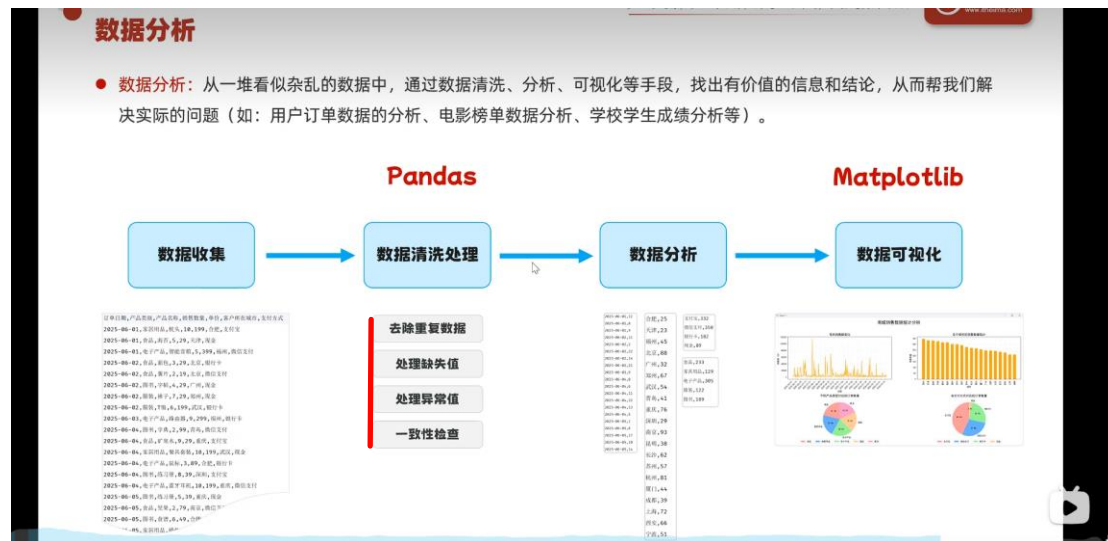


1.0 数据分析概述



环境准备:

Jupyter Notebook

- **Jupyter Notebook**是一个基于Web网页的、交互式的编程笔记本, 让你可以把代码、运行结果、图表和笔记全部都放在一个文件里 (在数据分析、机器学习、教学和科研等领域的数据实验室)。

快捷键:

单元格运行:ctrl+enter

新建单元格:shift+enter

Jupyter 在浏览器也可以打开;

笔记本中 默认输出最后一行(即只有最后一行不用写 print)

2.0 pandas 基础

Pandas介绍

- Pandas是一个功能强大的结构化数据分析的工具集, 底层是基于Numpy构建的, 无论是在数据分析领域、还是大数据开发场景中都有显著的优势。
- 官网: <https://pandas.pydata.org>
- 核心: DataFrame (类似表格)、Series (类似表格中的一列)

DataFrame

	Name	Age	Sex
0	Jay	18	male
1	Leo	22	male
2	Lily	25	female
3	Jerry	19	male
4	Cici	21	female

index column

Series

	Name
0	Jay
1	Leo
2	Lily
3	Jerry
4	Cici

index

	Age
0	18
1	22
2	25
3	19
4	21

index

- 需求：基于Pandas统计班级学员的各科成绩的最高分、最低分、平均分。

- 准备：npm install pandas==2.3.3

```
import pandas as pd

df1 = pd.DataFrame([
    {'姓名': '张三', '语文': 85, '数学': 92, '英语': 78},
    {'姓名': '李四', '语文': 78, '数学': 88, '英语': 95},
    {'姓名': '王五', '语文': 92, '数学': 96, '英语': 89},
    {'姓名': '赵六', '语文': 85, '数学': 90, '英语': 90},
    {'姓名': '孙七', '语文': 72, '数学': 59, '英语': 66},
    {'姓名': '周八', '语文': 80, '数学': 76, '英语': 68},
    {'姓名': '吴九', '语文': 85, '数学': 85, '英语': 85},
    {'姓名': '郑十', '语文': 57, '数学': 68, '英语': 49}
])

print(f"最高分: {df1['语文'].max()}, 最低分: {df1['语文'].min()}, 平均分: {df1['语文'].mean()}")
print(f"最高分: {df1['数学'].max()}, 最低分: {df1['数学'].min()}, 平均分: {df1['数学'].mean()}")
print(f"最高分: {df1['英语'].max()}, 最低分: {df1['英语'].min()}, 平均分: {df1['英语'].mean()}")
```

	姓名	语文	数学	英语
0	张三	85	92	78
1	李四	78	88	95
2	王五	92	96	89
3	赵六	85	90	90
4	孙七	72	59	66
5	周八	80	76	68
6	吴九	85	85	85
7	郑十	57	68	49

Npm(环境管理工具);下载稳定版本;

```
import pandas as pd

df=pd.DataFrame([ #列表 创建dataframe, 每个项为一行
    {"姓名":"贺晨浩","语文":100,"数学":90,"英语":80},
    {"姓名":"王伟伟","语文":90,"数学":57,"英语":76},
    {"姓名":"王伟伟","语文":90,"数学":80,"英语":54},
    {"姓名":"王伟伟","语文":76,"数学":90,"英语":100},
    {"姓名":"王伟伟","语文":73,"数学":43,"英语":68},
    {"姓名":"王伟伟","语文":45,"数学":35,"英语":90},
    {"姓名":"王伟伟","语文":45,"数学":90,"英语":63},
    {"姓名":"王伟伟","语文":76,"数学":80,"英语":74},
])

print(f"语文最高分:{df['语文'].max()},最低分:{df['语文'].min()},平均分:{df['语文'].mean()}")#df["语文"]直接获取列(series)
print(f"数学最高分:{df['数学'].max()},最低分:{df['数学'].min()},平均分:{df['数学'].mean()}")
print(f"英语最高分:{df['英语'].max()},最低分:{df['英语'].min()},平均分:{df['英语'].mean()}")
```

✓ [6] 353毫秒

语文最高分:100,最低分:45,平均分:74.375

数学最高分:90,最低分:35,平均分:70.625

英语最高分:100,最低分:54,平均分:75.625

2.1 Dataframe

DataFrame

- 构建DataFrame的方式有很多，主要方式如下：

```
df1 = pd.DataFrame([
    {'姓名': '张三', '语文': 85, '数学': 92, '英语': 78},
    {'姓名': '李四', '语文': 78, '数学': 88, '英语': 95},
    {'姓名': '王五', '语文': 92, '数学': 96, '英语': 89}
])

df2 = pd.DataFrame({
    '姓名': ['张三', '李四', '王五'],
    '语文': [85, 78, 92],
    '数学': [92, 88, 96],
    '英语': [78, 95, 89]
})

df3 = pd.DataFrame([
    ('张三', 85, 92, 78),
    ('李四', 78, 88, 95),
    ('王五', 92, 96, 89)
], columns=['姓名', '语文', '数学', '英语'])

df4 = pd.DataFrame([
    ['张三', 85, 92, 78],
    ['李四', 78, 88, 95],
    ['王五', 92, 96, 89]
], columns=['姓名', '语文', '数学', '英语'], index=['a', 'b', 'c'])
```



三种创建方式:1>列表 2>字典 3>指定列(用元组/列表)

1>dataframe 的增:

+ 增加新列 (最常用)

```
python

# 直接赋值 (推荐)
df['年薪'] = df['工资'] * 12

# 用 .insert() 插入到指定位置 (如第一列)
df.insert(0, '编号', [1, 2, 3])
```

复制 下载

+ 增加新行

```
python

# 方法1: 用 loc 追加 (如果索引不存在)
df.loc['新行索引'] = [值1, 值2, 值3]

# 方法2: 用 concat 拼接 (更安全, 推荐)
new_row = pd.DataFrame({'姓名': ['赵六'], '年龄': [35], '工资': [12000]})
df = pd.concat([df, new_row], ignore_index=True) # ignore_index 重置数字索引
```

将行拼接;

复制 下载

2>dataframe 的删:

— 删除列 (两种方式)

```
python

# 方式1: drop (不修改原数据, 默认返回新副本)
df = df.drop(columns=['工资']) # 删除工资列

# 方式2: del (直接修改原数据, 删除成功)
del df['工资'] # 简单粗暴, 常用于临时清理
```

复制 下载

— 删除行 (按索引)

```
python

# 删除索引为 0 和 2 的行
df = df.drop(index=[0, 2])
```

复制 下载

3>dataframe 的改:

✎ 修改单个单元格 (最高频场景)

```
python

# 推荐写法: 修改第0行 '年龄' 列的值
df.loc[0, '年龄'] = 18

# 如果明确知道位置, 用 at (比 loc 更快)
df.at[0, '年龄'] = 18
```

复制 下载

✂ 修改整列数据 (向量化操作)

```
python

# 把年龄全部加 1
df['年龄'] = df['年龄'] + 1

# 根据条件修改 (例如将年龄大于50的标记为“老年”)
df['年龄段'] = df['年龄'].apply(lambda x: '老年' if x > 50 else '青年')
```

复制 下载

✂ 修改列名

```
python

df = df.rename(columns={'姓名': '名字', '工资': '月薪'})
```

复制 下载

4>dataframe 的查:

场景	代码写法	返回类型	备注
查一列	<code>df['年龄']</code>	Series	✗ 绝对不要用 <code>df.年龄</code> (易与属性冲突), 推荐方括号
查多列	<code>df[['年龄', '工资']]</code>	DataFrame	⚠ 注意双括号, 少一个括号就是 Series
查一行 (按索引标签)	<code>df.loc['行标签']</code>	Series	适合索引是字符串/日期时
查一行 (按位置序号)	<code>df.iloc[0]</code>	Series	适合想取第几行 (从 0 开始)
查单元格 (标签定位)	<code>df.loc[行标签, '列名']</code>	Python 标量	最推荐, 清晰明了
查单元格 (位置定位)	<code>df.iloc[0, 0]</code>	Python 标量	极快, 但依赖行数列数
条件筛选 (返回多行)	<code>df[df['年龄'] > 25]</code>	DataFrame	你之前用的 <code>df3[df3 > 阈值]</code> 就是这个原理
条件筛选+取特定列	<code>df.loc[df['年龄'] > 25, ['姓名', '年龄']]</code>	DataFrame	推荐写法: 先筛选, 再取列

5>dataframe 的常见方法:

```
# DataFrame 常见属性 - index , columns , values , size , dtypes , shape
# df4.index.tolist() # index : 索引
# df4.columns.tolist() # columns : 列名
# df4.values.tolist() # values : 值
# df4.size # size : 元素个数
# df4.dtypes # dtypes : 数据类型
df4.shape # shape : 维度(行, 列)
```

获取属性不加括号;直接返回的是 object,可用 tolist()转为列表;

.values 将每行数据返回;tolist 后返回二维列表;

dtypes 查看每列类型(为 numpy)中的类型而非 py;

2.2 Series

Series

- 构建Series的方式有很多, 主要方式如下:

```
s = pd.Series([10, 20, 30, 40, 50])

s = pd.Series([10, 20, 30, 40, 50], index=['a', 'b', 'c', 'd', 'e'])

s = pd.Series({'a': 10, 'b': 20, 'c': 30, 'd': 40, 'e': 50})

s = df1['语文']

s = df1['数学']
```

a	10
b	20
c	30
d	40
e	50

s.index: 获取索引

s.values: 获取值

s.dtype: 获取数据类型

s.size: 获取数据个数

s.shape: 获取数据维度(行,)

用元组,列表,字典都可以创建;

增删改查和 dataframe 只有维度上的区别;

因为 series 只有一列,所以 dtype 只有一个值;

series 和 dataframe 中的一列都可以进行聚合运算:

Series 支持几乎所有聚合计算, 用法都和你写的 `.sum()` 一样:

- `df3.mean()` (平均值)
- `df3.max()` (最大值)
- `df3.min()` (最小值)
- `df3.median()` (中位数)
- `df3.count()` (非空值个数, 注意它和 `len()` 的区别)

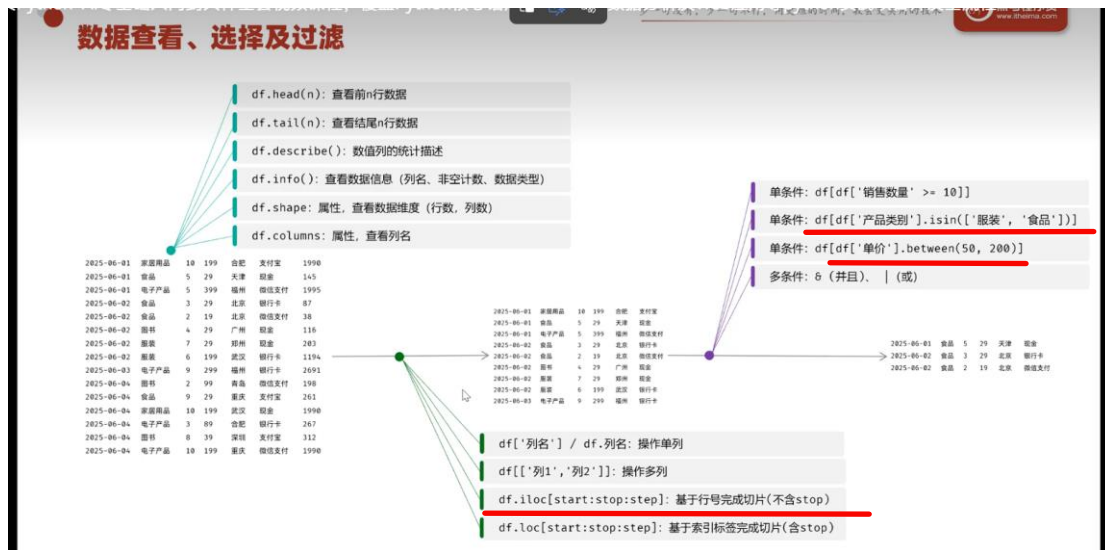
2.3 pd 中数据的读取和写入

```
df=pd.read_csv('data/movies.csv',usecols=['电影id','评分'])
df['真实评分']=df['评分']-50
df.to_csv('data/movies_new.csv',index=False)
✓ [10] 25毫秒
```

Usecols:加载哪些列;

Index=false:不写入索引

2.4 pd 中数据的查看,选择,过滤



数据的查看,选择返回的都是 dataframe 格式

数据查看:

```
1 df1.head(10) # 显示前10行数据
2 df1.tail(10) # 显示最后10行数据
3 df1.describe() # 显示数据统计信息
4 df1.info() # 显示数据信息(列名, 非空计数, 数据类型等)
5 df1.shape # 显示数据行数和列数
6 df1.columns # 显示列名
```

数据选择:(列,行)

```
1 # 1. 选择列
2 # 1.1 单列
3 # df2['产品名称']
4 # df2.产品名称
5
6 # 1.2 多列
7 # df2[['产品名称', '单价']]
8 # df2[['产品类别', '产品名称', '单价']]
9
10 # 2. 选择行 - iloc, loc
11 # 2.1 iloc ----> 基于行号选择行 (不包含结束位置), 语法: df.iloc[start:stop:step]
12 # df2.iloc[0:5:1]
13 # df2.iloc[0:5]
14
15 # 2.2 loc ----> 基于索引选择行 (包含结束位置), 语法: df.loc[start:stop:step]
16 df2.loc[6805677496:5888085066:2]
```

iloc 是根据行号,loc 是根据索引;注意行号和索引的区别:行号不会改变 0-n

数据选择:

```
df3 = pd.read_csv('data/sales.csv')

# 数据过滤 ---> df[filter]
# 1. 获取 销售数量 >= 10 的订单数据
df3[df3['销售数量'] >= 10]

# 2. 获取产品类别为 食品 或 图书 的订单数据
df3[df3['产品类别'].isin(['食品', '图书'])]

# 3. 获取 单价在100-200之间 的订单数据 ---> 范围: between ---> 默认包含边界 (inclusive="left")
df3[df3['单价'].between(79, 199)]

# 4. 获取 销售数量 >= 8 , 并且 单价 >= 100 的订单数据 ----> 多条件(并且: & , 或者: |)
df3[(df3['销售数量'] >= 8) & (df3['单价'] >= 100)]

# 5. 获取 产品类别为 服装/食品 , 支付方式为 支付宝/微信支付 的订单数据
df3[(df3['产品类别'].isin(['服装', '食品'])) & (df3['支付方式'].isin(['支付宝', '微信支付']))]
```

先用整列和值比较,返回 bool 列表,再选出来;而不是单个数据的选;

```
# 配置项, 展示所有的数据
# df.set_option('display.max_rows', None)
```

2.5 数据清洗

数据清洗

- 数据清洗是指发现并纠正数据中可识别的错误的过程, 包括处理缺失值、重复值、异常值, 统一数据格式, 保证数据的一致性。

	订单号	产品类别	产品名称	销售数量	单价	客户所在城市	支付方式	订单日期	
	0	6805677496	家居用品	枕头	10	199	合肥	支付宝	2025-06-01
	1	5857234132	食品	海苔	5	29	天津	现金	2025-06-01
重复值处理	2	5888085059	电子产品	智能音箱	5	399	福州	微信支付	2025-06-01
	3	6823473137	食品	面包	3	29	北京	银行卡	2025-06-02
	4	6801664364	食品	薯片	2	19	NaN	微信支付	2025/06/02
	5	5897064217	图书	字帖	4	29	广州	现金	2025-06-02
	6	6857752388	服装	袜子	7	-29	郑州	现金	2025-06-02
	7	6898807429	服装	T恤	6	199	武汉	银行卡	2025-06-02
	8	5875402232	电子产品	路由器	9	299	福州	银行卡	2025-06-03
	9	5888085059	图书	字典	2	99	青岛	微信支付	2025-06-04

缺失值处理

异常格式处理

异常值处理

```
pd.set_option('display.min_rows', 30) # 设置显示行数(默认10行)
```

2.5.1 缺失值处理

```
# 2. 数据清洗
# 2.1 缺失值处理
# df.isnull() # 查看缺失值

# 2.1.2 填充缺失值
# df.fillna('--') # 填充缺失值
# df.fill() # 填充缺失值, 填充上一行数据
df.bfill() # 填充缺失值, 填充下一行数据
```


isnull 返回所有 bool,为 True 的位置为 NaN;

2.5.2 重复值处理

```
1 # 2.2 重复值处理
2 # 2.2.1 查看重复值
3 df.duplicated() # 查看重复值(所有的列的数据都重复)
4 df.duplicated(subset=['订单号']) # 查看重复值(指定列的数据重复)
5
6 # 2.2.2 删除重复值
7 df.drop_duplicates(subset=['订单号'], keep='last') # keep='last' 保留重复值最后一行; keep='first' 保留重复值第一行;
```

Duplicated:即两行每列值都相同时,才会判定为重复;设置 subset 为根据指定列判定;

2.5.3 异常值处理

```
1 # 2.3 异常值处理
2 # 2.3.1 查看异常值
3 df[df['单价'] < 0]
4
5 # 2.3.2 删除异常值
6 df.drop(df[df['单价'] < 0].index)
7
8 # 2.3.3 修复异常值
9 df['单价'] = df['单价'].abs() # 绝对值
10 df
```

数据选择出来的 pd 的子集也可以用 pd 的属性访问方法;

2.5.4 数据格式处理

```
1 # 2.4 数据格式处理
2 df['订单日期'] = df['订单日期'].str.replace('/', '-')
3 df
```

.str 将数据转为 py 的字符串,即可根据 String 的方法进行替换;

2.6 数据排序

数据排序

- 在进行数据排序时,有两种排序方式,分别是:升序和降序。而基于Pandas进行数据排序时,是可以按照多个列进行排序的。

降序: df.sort_values('销售量', ascending=False)

订单号	产品类别	产品名称	销售量	单价
0	6805677496	家居用品	枕头	10 199
1	5857234132	食品	海苔	5 29
2	5888085059	电子产品	智能音箱	5 399
3	6823473137	食品	面包	3 29
4	6801664364	食品	薯片	2 19
5	5897064217	图书	字帖	4 29
6	6857752388	服装	袜子	7 29
7	6898807429	服装	T恤	6 199
8	5875402232	电子产品	路由器	9 299
9	5888085066	图书	字典	2 99

升序: df.sort_values('销售量', ascending=True)

df.sort_values(['销售量', '单价'], ascending=[True, True])

多列排序: 进行多个列排序时,会先按照第一列进行排序,第一列的值相同时,才会按照第二列进行排序。

2.7 数据分组

数据分组

- 分组操作就是把数据按照某个特征分成不同的组，然后对每个组分别进行统计计算。

订单号	产品类别	产品名称	销售数量	单价	销售额
0	家居用品	枕头	10	199	1990
1	食品	海苔	5	29	145
2	电子产品	智能音箱	5	399	1995
3	食品	面包	3	29	87
4	食品	薯片	2	19	38
5	图书	字帖	4	29	116
6	服装	袜子	7	29	203
7	服装	T恤	6	199	1194
8	电子产品	路由器	9	299	2691
9	图书	字典	2	99	198

订单号	产品类别	产品名称	销售数量	单价	销售额
0	家居用品	枕头	10	199	1990
订单号	产品类别	产品名称	销售数量	单价	销售额
1	食品	海苔	5	29	145
3	食品	面包	3	29	87
4	食品	薯片	2	19	38
订单号	产品类别	产品名称	销售数量	单价	销售额
2	电子产品	智能音箱	5	399	1995
8	电子产品	路由器	9	299	2691
订单号	产品类别	产品名称	销售数量	单价	销售额
5	图书	字帖	4	29	116
9	图书	字典	2	99	198
订单号	产品类别	产品名称	销售数量	单价	销售额
6	服装	袜子	7	29	203
7	服装	T恤	6	199	1194

分组后求和 `df.groupby('产品类别')['销售额'].sum()`
分组后统计数量 `df.groupby('产品类别')['销售额'].count()`
分组后求最大值 `df.groupby('产品类别')['销售额'].max()`
分组后求最小值 `df.groupby('产品类别')['销售额'].min()`
分组后求平均值 `df.groupby('产品类别')['销售额'].mean()`

`df.groupby('产品类别')['销售额'].agg(['sum', 'count', 'max', 'min', 'mean'])`

3.7 根据 产品类别 分组，统计各个类别的商品的 平均单价、最高单价、最低单价 - agg
`df.groupby('产品类别')['单价'].agg(['mean', 'max', 'min'])`

3.8 根据 产品类别 分组，统计各个类别的商品的 销售数量 之和，销售金额 之和，平均 单价
`df.groupby('产品类别').agg({'销售数量': 'sum', '销售金额': 'sum', '单价': 'mean'})`

3.7 根据 产品类别 分组，统计各个类别的商品的 平均单价、最高单价、最低单价 - agg
`df.groupby('产品类别')['单价'].agg(['mean', 'max', 'min'])`
`df.groupby('产品类别').agg({'单价': ['mean', 'max', 'min']})`

返回：分组值+要求统计数据 的 dataframe

3.0 Matplotlib 基础

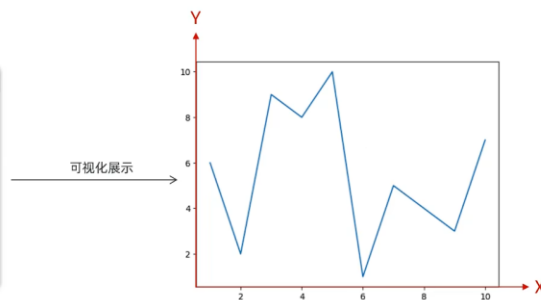
Matplotlib初识

- Matplotlib是一个功能强大的数据可视化开源Python库，也是Python中使用的最多的图形绘图库，可以创建静态、动态、交互式的图表。
- 官网: <https://matplotlib.org>
- 安装: `pip install matplotlib`

```
import matplotlib.pyplot as plt

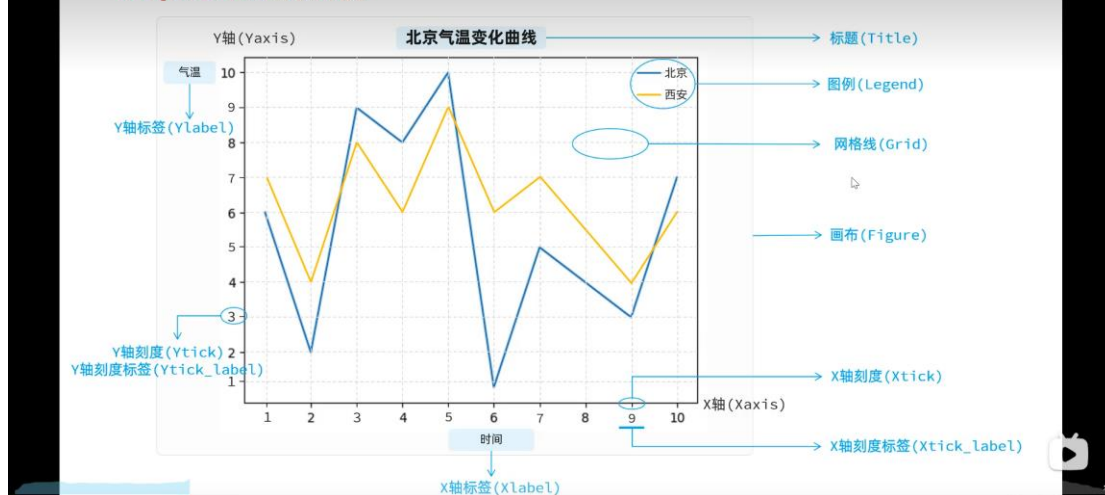
x = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
y = [6, 2, 9, 8, 10, 1, 5, 4, 3, 7]

plt.plot(x, y)
plt.show()
```



注意：X轴的数据数量与Y轴中的数据数量要一致。

Matplotlib图表详解



```
import matplotlib.pyplot as plt
import random

from matplotlib.lines import LineStyle

plt.rcParams['font.sans-serif']=['SimHei'] #设置中文以黑体输出
✓ [42] 18毫秒
```

```
x=[1,2,3,4,5,6,7,8,9,10]#x轴坐标
y_beijing=[ random.randint(10,15) for i in x]#北京y轴坐标
y_xian=[ random.randint(10,15) for i in x]#西安y轴坐标

plt.figure(figsize=(10,5))#创建画布,并设置大小
plt.plot(x,y_beijing,label='北京')#绘制,若上面不自己创建画布,则此处自动创建
plt.plot(x,y_xian,label='西安')
#设置标题
plt.title("气温变化图",fontsize=13)
#设置xy轴标签
plt.xlabel("时间")
plt.ylabel("温度")
#设置xy轴刻度
y_tick=[i for i in range(9,16)]
plt.xticks(x[::1])
plt.yticks(y_tick)
#设置网格,alpha为透明度0-1
plt.grid(linestyle='-.',alpha=0.3)
#设置图例(Label在绘制时指定)
plt.legend()
#展示图表
plt.show()
✓ [46] 121毫秒
```

Plt 自动化工具,对象都自动创建,故调用时不用对象.的形式

创建子图

- 为了能同时展示多个图表,便于图表之间数据的直观对比和分析,更高效、更专业地组织和呈现复杂的可视化信息,通常会在一个画布上创建多个子图。

```
figure, axes = plt.subplots(nrows=2, ncols=2, figsize=(20, 6), dpi=100)
```

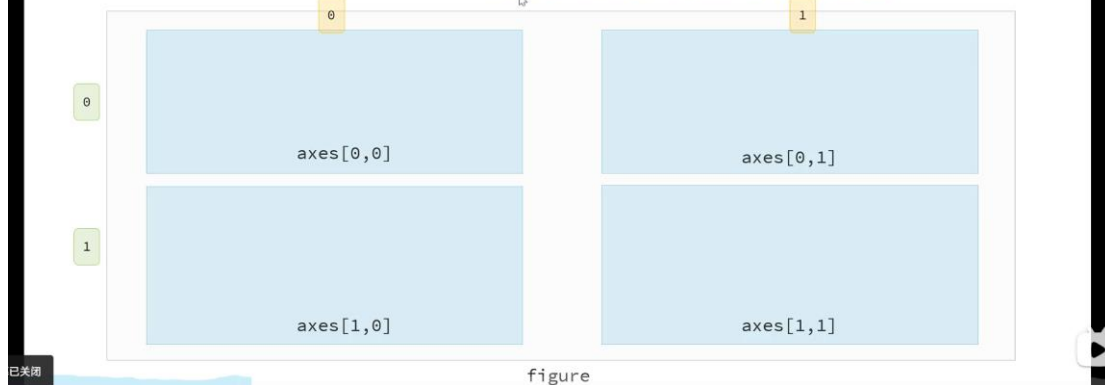
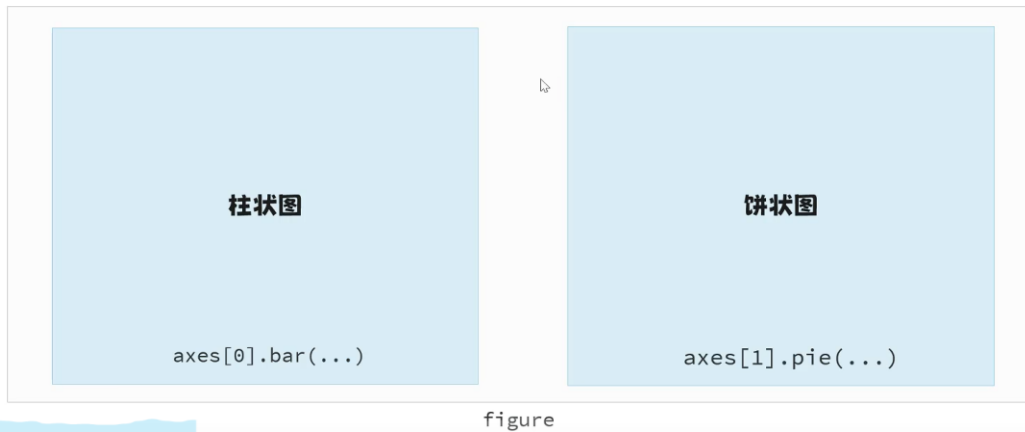


Figure 为整个画布;axes 为图的数组;dpi 为分辨率;

```
figure, axes = plt.subplots(nrows=1, ncols=2, figsize=(20, 6), dpi=100)
```



柱状图画法:

```
import matplotlib.pyplot as plt
from matplotlib.axes import Axes

plt.rcParams['font.sans-serif']=['SimHei'] #设置中文以黑体输出

#创建子图:figure为画布对象;axes为子图数组,Axes类型
figure, axes=plt.subplots(nrows=1,ncols=2,figsize=(20,6),dpi=100)

countries=['中国','美国','英国','法国','德国','日本','印度']
values=[45,30,25,20,15,10,5]

axes1: Axes=axes[0] #重命名,加上注解以便提示信息
axes1.bar(countries,values,width=0.6,color='g') #创建柱状图,设置宽度和颜色
axes1.set_title('世界石油储备',fontsize=16)
axes1.set_xlabel('国家',fontsize=13)
axes1.set_ylabel('石油储备(亿吨)',fontsize=13)

axes1.grid(linestyle='--',alpha=0.3)
```

饼状图画法:

```
4 countries2=['中国','美国','英国','法国','德国','日本','印度']
5 values2=[45,30,25,20,15,10,5]
6
7 axes2: Axes=axes[1]
8 axes2.pie(values2,labels=countries2,autopct='%1.1f%%') #中间符号为显示,两侧为格式;前1为最小长度(不够前面添空格),后1为保留小数位;
9 axes2.set_title('世界石油储备占比',fontsize=16)
0 axes2.legend(loc='lower center',ncol=5,bbox_to_anchor=(0.5,-0.08)) #ncol设置图例显示列数;
1 #bbox:0.5表示左右中间,-0.08表示上下方向-0.08的位置;
2
3 plt.show()
```